

# Information Retrieval Models — Vector Space and Probabilistic Approaches

Muhammad Bagus Zulmi (25/572328/PPA/07191)

Vicky Mahfudy (25/573019/PPA/07207)

## 1 Pendahuluan

Laporan ini membandingkan performa 6 model *ranked retrieval* pada korpus 14.343 dokumen berita berbahasa Indonesia (`News.csv`, kolom `content`): TF, TF-IDF, dan Word2Vec (*TF-IDF weighted*) sebagai model *vector space*; serta Query-Likelihood (QL) tanpa *smoothing*, QL Laplace, dan QL *linear interpolation* ( $\lambda = 0,5$ ) sebagai model probabilistik. Preprocessing yang diterapkan pada semua model: *lowercasing*, penghapusan tanda baca, tokenisasi, dan penghapusan *stopword* Bahasa Indonesia. Kelima query uji yang sama digunakan pada seluruh model, masing-masing mengambil *top-10* dokumen. Kode dan hasil lengkap tersedia di <https://github.com/vickymahfudy/stbi-tugas-1>.

## 2 Waktu Komputasi

| Model                      | Mean (s) | Std (s) | Min (s) | Max (s) |
|----------------------------|----------|---------|---------|---------|
| Word2Vec (TF-IDF weighted) | 0,0046   | 0,0023  | 0,0029  | 0,0086  |
| QL (No Smoothing)          | 0,0087   | 0,0029  | 0,0060  | 0,0136  |
| TF-IDF                     | 0,0130   | 0,0030  | 0,0101  | 0,0171  |
| QL (Laplace)               | 0,0208   | 0,0057  | 0,0148  | 0,0301  |
| QL (Linear Interpolation)  | 0,0248   | 0,0049  | 0,0163  | 0,0290  |
| TF                         | 0,0624   | 0,0084  | 0,0569  | 0,0771  |

Table 1: Rata-rata waktu komputasi per model (5 query, korpus 14.343 dokumen).

Word2Vec paling cepat karena *similarity* dihitung pada ruang vektor berdimensi rendah (100) yang sudah dibangun sebelumnya. QL tanpa *smoothing* juga cepat karena dokumen langsung tereliminasi (skor  $-\infty$ ) begitu satu kata query tidak ditemukan. TF paling lambat karena implementasi `CountVectorizer` tidak melakukan *precomputation* bobot seperti TF-IDF, membuat *transform* dan *cosine similarity* lebih mahal.

## 3 Relevansi Dokumen Terhadap Query

Sebagai proksi otomatis relevansi digunakan `term_overlap_ratio` (rasio kata query yang muncul literal di dokumen hasil), dirangkum pada Tabel 2.

| Model                      | Mean  | Std   |
|----------------------------|-------|-------|
| QL (No Smoothing)          | 1,000 | 0,000 |
| QL (Linear Interpolation)  | 0,931 | 0,094 |
| QL (Laplace)               | 0,909 | 0,113 |
| TF                         | 0,781 | 0,190 |
| TF-IDF                     | 0,754 | 0,203 |
| Word2Vec (TF-IDF weighted) | 0,698 | 0,220 |

Table 2: Rata-rata proksi relevansi (`term_overlap_ratio`) per model.

Nilai 1,000 pada QL tanpa *smoothing* bukan tanda relevansi terbaik, melainkan artefak mekanisme kerjanya: dokumen yang tidak memuat seluruh kata query otomatis dieliminasi, sehingga yang lolos filter pasti *overlap* penuh, dan *top-10*-nya seringkali tidak terisi penuh akibat *data sparsity*. Sebaliknya, nilai Word2Vec yang paling rendah menandakan model ini menangkap kemiripan semantik, bukan berarti paling tidak relevan. Evaluasi relevansi yang valid membutuhkan penilaian manual terhadap isi dokumen, bukan hanya *overlap* kata literal.

## 4 Perbandingan Top-10 Antar Model

*Jaccard similarity* dari himpunan dokumen *top-10* dihitung untuk setiap pasangan model pada masing-masing dari 5 query, lalu dirata-ratakan (Tabel 3).

| Pasangan Model                       | Jaccard (rata-rata) |
|--------------------------------------|---------------------|
| QL No Smoothing vs QL Linear Interp. | 0,579               |
| TF vs QL Linear Interp.              | 0,271               |
| TF vs TF-IDF                         | 0,251               |
| TF-IDF vs QL Linear Interp.          | 0,247               |
| TF vs Word2Vec                       | 0,209               |
| QL No Smoothing vs QL Laplace        | 0,197               |
| TF vs QL No Smoothing                | 0,172               |
| QL Laplace vs QL Linear Interp.      | 0,168               |
| TF-IDF vs Word2Vec                   | 0,161               |
| TF-IDF vs QL No Smoothing            | 0,154               |
| TF vs QL Laplace                     | 0,153               |
| TF-IDF vs QL Laplace                 | 0,153               |
| Word2Vec vs QL Linear Interp.        | 0,133               |
| Word2Vec vs QL No Smoothing          | 0,098               |
| Word2Vec vs QL Laplace               | 0,093               |

Table 3: Rata-rata *Jaccard similarity* top-10 antar seluruh pasangan model (5 query).

QL tanpa *smoothing* dan QL *linear interpolation* paling mirip (Jaccard 0,579), wajar karena keduanya berbasis probabilitas kata yang sama sebelum *smoothing* diterapkan. Word2Vec konsisten memiliki *overlap* terendah dengan seluruh varian QL (0,09–0,13), mengonfirmasi representasi semantik menangkap relevansi dari sudut pandang berbeda dibanding model probabilistik berbasis kata literal.

## 5 Efek Smoothing dan Normalisasi

### 5.1 Efek Smoothing pada Model QL

Tabel 4 menunjukkan rata-rata jumlah dokumen yang berhasil dikembalikan (dari target *top-10*) oleh masing-masing varian QL, dirata-ratakan atas 5 query.

| Varian QL            | Rata-rata Hasil | Min | Max |
|----------------------|-----------------|-----|-----|
| Tanpa Smoothing      | 8,0             | 4   | 10  |
| Laplace              | 10,0            | 10  | 10  |
| Linear Interpolation | 10,0            | 10  | 10  |

Table 4: Rata-rata jumlah dokumen hasil (dari target top-10) per varian QL (5 query).

QL tanpa *smoothing* pada 2 dari 5 query hanya mengembalikan sebagian dokumen (bahkan hingga 4 dari 10) karena mekanismenya mengeliminasi total dokumen yang tidak memuat *seluruh* kata query (skor  $-\infty$ ). *Smoothing*, baik Laplace maupun *linear interpolation*, menghilangkan masalah ini sepenuhnya: probabilitas kata yang tidak ada di dokumen tidak lagi nol, sehingga *top-10* selalu terisi penuh pada seluruh query. Sebagai konsekuensinya, seperti terlihat pada Tabel 2, proksi relevansi rata-rata sedikit menurun setelah *smoothing* ( $1,000 \rightarrow 0,909$  untuk Laplace,  $0,931$  untuk *linear interpolation*), karena dokumen yang dulunya tereliminasi (dan pasti *overlap* penuh bila lolos) kini ikut masuk peringkat meski *overlap* kata-nya tidak lengkap. *Trade-off* ini wajar: *smoothing* menukar sedikit presisi literal demi *recall* yang jauh lebih baik.

### 5.2 Efek Normalisasi (Stemming, Stopword Removal, Lowercasing)

Seluruh model pada laporan ini sudah menerapkan *lowercasing* dan penghapusan *stopword* Bahasa Indonesia sebagai *preprocessing* dasar (Bagian 1). Kami mencoba menambahkan langkah *stemming* (Sastrawi) di atas model TF-IDF sebagai *enhancement* lanjutan, namun proses *stemming* pada *vocabulary* unik dari 14.343 dokumen berjalan lebih dari 15 menit tanpa selesai (Sastrawi murni Python, lambat secara *inherent* per kata) sehingga dihentikan sebelum menghasilkan data. Akibatnya, efek *stemming* terhadap performa retrieval belum dapat dianalisis secara empiris pada laporan ini; evaluasi ini menjadi pekerjaan lanjutan, dengan kandidat solusi berupa *caching* hasil *stemming* ke *disk* atau membatasi lingkup *stemming* pada subset korpus yang lebih kecil.